Course: MATH 373 – Statistical Learning
Team Members: Disha Khati, Tyler Soong, Kailash Turimella
Professor: Dr. James D Wilson
Date: May 10th, 2026

QPDAC Report

Sentiment Analysis on IMDB Movie Reviews

1. Motivation

As frequent users of Letterboxd, we became interested in how the language of movie reviews encodes sentiment. Letterboxd reviews are informal, expressive, and highly varied in tone and vocabulary, which makes them rich and challenging text data to work with.

We wanted to go beyond simple keyword matching and explore what unsupervised topic modeling could reveal about how people talk about films, and whether those latent themes carry predictive signals for sentiment classification. This project gave us the opportunity to combine text feature engineering (TF-IDF and LDA) with classical machine learning classifiers in a way that is both analytically principled and practically motivated.

2. QPDAC Analysis Plan

Question

Can we predict whether an IMDB movie review is positive or negative using topic modeling (LDA) and term frequency features (TF-IDF)? And what do the latent topics discovered by LDA reveal about the language and themes reviewers use when discussing films?

Plan

We designed a three-stage pipeline:

- ***Text preprocessing***: strip HTML artifacts, remove punctuation, lowercase, lemmatize, and remove stopwords to produce clean review text.
- ***Feature engineering***: extract two parallel feature representations, TF-IDF (lexical) and LDA topic proportions (semantic), and test them individually and combined.
- ***Classification***: train multiple Naive Bayes variants and Random Forest classifiers on each feature set, then compare performance using accuracy, F1-score, and confusion matrices.

Data

***Source*:** IMDB Dataset of 50,000 Movie Reviews (Kaggle).

***Structure*:** 50,000 reviews with binary sentiment labels (positive/negative), perfectly balanced at 25,000 each.

***Key challenges*:** Raw reviews contain HTML artifacts (e.g.,
), high-dimensional sparse vocabulary after vectorization, and subjective LDA topic interpretation.

Analysis

We compared seven model configurations across two classifier families:

- ***Naive Bayes***: ComplementNB on TF-IDF, MultinomialNB on raw counts, GaussianNB on LDA features, and ComplementNB on the combined TF-IDF + LDA matrix.

- ***Random Forest***: applied to TF-IDF, LDA, and combined feature sets separately.
- ***Vectorizer comparison***: CountVectorizer vs. TfidfVectorizer using the same classifier (ComplementNB) to isolate the effect of representation.

Conclusion

TF-IDF-based models consistently outperformed LDA-only models, with the best overall result achieved by the TF-IDF + LDA combined Random Forest (86.44%). Naive Bayes with LDA features alone achieved 79.51%, illustrating the information loss from compressing 10,000 vocabulary dimensions down to 10 topic proportions. Random Forest was better able to exploit the combined feature space than Naive Bayes, which is our most analytically interesting finding. Full results and discussion are presented in subsequent sections.

3. Methods and Pipeline

3.1 Text Cleaning

Raw IMDB reviews required significant preprocessing before feature extraction. We applied the following steps in order:

- HTML tag removal using regex (e.g.,
 artifacts common in the IMDB dataset).
- Punctuation and numeric character removal.
- Lowercasing all text.
- Stopword removal using NLTK's English stopword list.
- Lemmatization using NLTK's WordNetLemmatizer to reduce words to base form (e.g., "running" → "run", "movies" → "movie").

Challenge: Lemmatization without part-of-speech tagging defaults to noun lemmatization, which means verb forms may not be correctly reduced (e.g., "was" stays "was" rather than "be"). Full POS-tagged lemmatization would improve quality but adds significant computation time over 50,000 reviews.

3.2 Feature Engineering

TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF weights each word by how often it appears in a document relative to how commonly it appears across all documents. Words that are frequent in one review but rare globally (e.g., "cinematography") receive high weight, while ubiquitous words (e.g., "the") receive low weight regardless of how often they appear. We used unigrams and bigrams with a vocabulary cap of 10,000 features, producing a sparse float matrix of shape (40,000 × 10,000) for training.

LDA(Latent Dirichlet Allocation)

LDA is an unsupervised topic modeling algorithm that models each document as a mixture of latent topics, where each topic is a probability distribution over words. Given a corpus, it infers what topics must have generated the observed word patterns. LDA requires raw integer counts rather than TF-IDF weights, as it relies on word frequency probabilities to estimate topic distributions.

We initially fit 15 topics on the training corpus. However, we found that too many topics introduced noise - individual words dominated single topics and the clusters became difficult to interpret meaningfully. For example, generic words like "film" and "movie" skewed entire topics, obscuring the thematic structure we were trying to capture. Reducing to 10 topics produced cleaner, more coherent clusters and improved downstream classification accuracy from 77.35% to 79.51% with GaussianNB.

Each review is represented as a 10-dimensional vector of topic proportions - a semantic fingerprint of which themes the review covers. Looking at our fitted 10-topic model, we can interpret several clusters by hand:

- **Topic 6:** "film, one, horror, good, like, woman, house, also, role, scene" - clusters around horror film reviews
- **Topic 2:** "movie, game, one, good, action, get, great, fight, scene, like" - captures action and sports film reviews
- **Topic 3:** "one, film, get, john, play, song, role, movie, star, musical" - reflects musical and performance reviews
- **Topic 8:** "film, life, one, war, story, family, man, young, woman, world" - identifies war and family drama reviews
- **Topic 5:** "show, one, movie, time, like, series, episode, year, tv, great" - clearly groups TV show reviews

This experience - trying 15 topics, observing the degradation in interpretability, and reducing to 10 - is itself an important methodological lesson: in unsupervised topic modeling, more topics is not always better. The number of topics is a hyperparameter that requires empirical validation, which we guided using a perplexity curve.

These topic clusters are what LDA brings to the project beyond classification - they tell us something meaningful about the structure of how people write about movies, independent of sentiment.

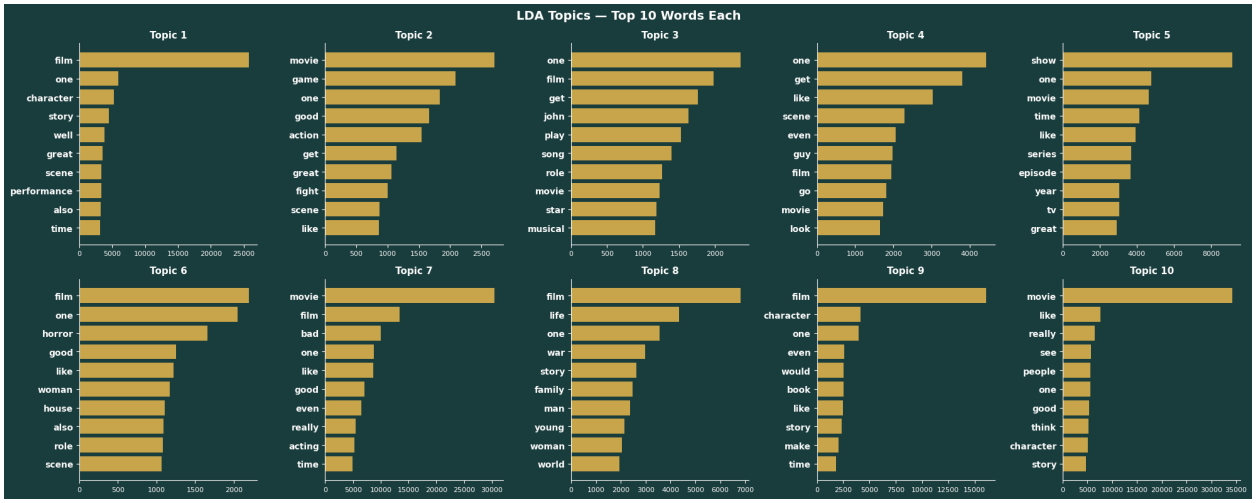

Figure 1: LDA Topics — Top 10 words per topic across all 10 discovered clusters.

3.3 Chi-squared Feature Selection

While TF-IDF measures how important a word is within a single review, it does not tell us how useful that word is for predicting sentiment globally. We used chi-squared testing to measure how strongly each word in the TF-IDF vocabulary is associated with positive or negative sentiment across the entire training set.

The null hypothesis is that a word's frequency is independent of sentiment - its presence carries no information about whether a review is positive or negative. A high chi-squared score indicates strong grounds to reject this null hypothesis, meaning the word appears at significantly different rates across the two classes.

Notable findings from the top 20 most discriminative words: "bad" (251.65), "worst" (204.23), "waste" (149.14), "awful" (138.63), and "great" (126.59) - confirming that strongly evaluative words, both negative and positive, are the primary drivers of classification. Bigrams also featured prominently: "waste time" and "worst movie" ranked in the top 10, validating our decision to include bigrams in the TF-IDF vocabulary. Words with chi-squared scores of 0.0 - such as "leading", "exotic", and "fatal" - appear equally across both classes and contribute no discriminative signal despite having non-zero TF-IDF scores, illustrating that rarity alone does not imply usefulness for classification.

Note: all p-values underflow to zero due to sample size (n=40,000). The floating point representation cannot store values as small as 10^{-300}, so we interpret chi-squared scores directly as a measure of discriminative power rather than relying on p-values.

3.4 Classifiers

Naive Bayes Variants

ComplementNB: Used with TF-IDF features. Unlike standard MultinomialNB which estimates how likely a word is given one class, ComplementNB estimates how likely a word is given all other classes combined - the class whose complement probability is weakest wins. It handles continuous non-negative float features (like TF-IDF weights) more robustly than MultinomialNB, which was originally designed for integer counts.

MultinomialNB: Used with raw CountVectorizer output (integer word counts). This is the classic Naive Bayes formulation for text - it models word frequencies within each class and assumes features are multinomially distributed.

GaussianNB: Used with LDA topic proportion vectors (floats between 0 and 1 that sum to 1 per review). GaussianNB assumes features are normally distributed within each class, making it appropriate for the continuous topic proportions that LDA produces.

Random Forest

Random Forest is an ensemble method that trains multiple decision trees on random subsets of the data and aggregates their predictions by majority vote. Unlike Naive Bayes, it makes no assumptions about the distribution of input features, making it directly compatible with TF-IDF floats, LDA proportions, and the combined matrix using the same classifier. This allowed us to isolate the effect of feature representation independently of classifier choice.

3.5 Combined Features

We horizontally stacked the TF-IDF matrix (10,000 columns) and the LDA topic matrix (10 columns) using scipy's hstack to produce a combined feature matrix of 10,010 features per review. This was used

with both ComplementNB and Random Forest to test whether semantic LDA features add predictive signal on top of lexical TF-IDF features.

4. Unusual Challenges and How We Overcame Them

HTML Artifacts in Raw Text

The IMDB dataset contains HTML tags (most commonly
) embedded within review text - a byproduct of how reviews were scraped from the web. Left uncleaned, these appear as vocabulary tokens and pollute the feature space. We addressed this with a regex substitution step (re.sub(r'<.*?>', ' ', text)) at the start of the cleaning pipeline.

Vectorizer-Classifier Compatibility

A non-obvious challenge was that different classifiers require different input types. MultinomialNB requires non-negative integers, ComplementNB handles floats, and GaussianNB expects continuous values. This meant we could not use a single vectorizer across all models. We maintained two separate CountVectorizers (one for LDA, one for MultinomialNB) and one TfidfVectorizer, each fit only on the training set to prevent data leakage.

LDA Dimensionality and Classifier Compatibility

LDA outputs dense numpy arrays of floats, while TF-IDF produces sparse matrices. Combining them required wrapping the LDA output in csr_matrix() before passing to scipy's hstack. Additionally, feeding LDA topic proportions into MultinomialNB is invalid since the values are floats — requiring the use of GaussianNB instead.

Python 3.13 Multiprocessing Errors

Using n_jobs=-1 on LDA (to parallelize across CPU cores) produced verbose ResourceTracker errors on Python 3.13 due to a known bug in multiprocessing cleanup. We resolved this by setting n_jobs=1, accepting a modest speed tradeoff for clean output.

LDA and QDA Classifier Limitations

We attempted to use Linear Discriminant Analysis (LDA classifier) and Quadratic Discriminant Analysis (QDA) on the full TF-IDF feature matrix. Both require computing and inverting covariance matrices, which is computationally infeasible at 10,000 dimensions. Neither classifier supports sparse matrix input. After attempting dimensionality reduction via TruncatedSVD as a workaround, the results did not improve on simpler models and added significant complexity, so we excluded them from the final analysis. This experience reinforced that classifier choice must be informed by the geometry and dimensionality of the feature space.

5. Results

5.1 Model Accuracy Comparison

Model	Accuracy
TF-IDF + ComplementNB	**86.51%**
Counts + MultinomialNB	**85.78%**
LDA + GaussianNB	**79.51%**
TF-IDF + LDA Combined (NB)	**86.07%**
TF-IDF + Random Forest	**85.61%**
LDA + Random Forest	**81.82%**
TF-IDF + LDA Combined (RF)	**86.44%**

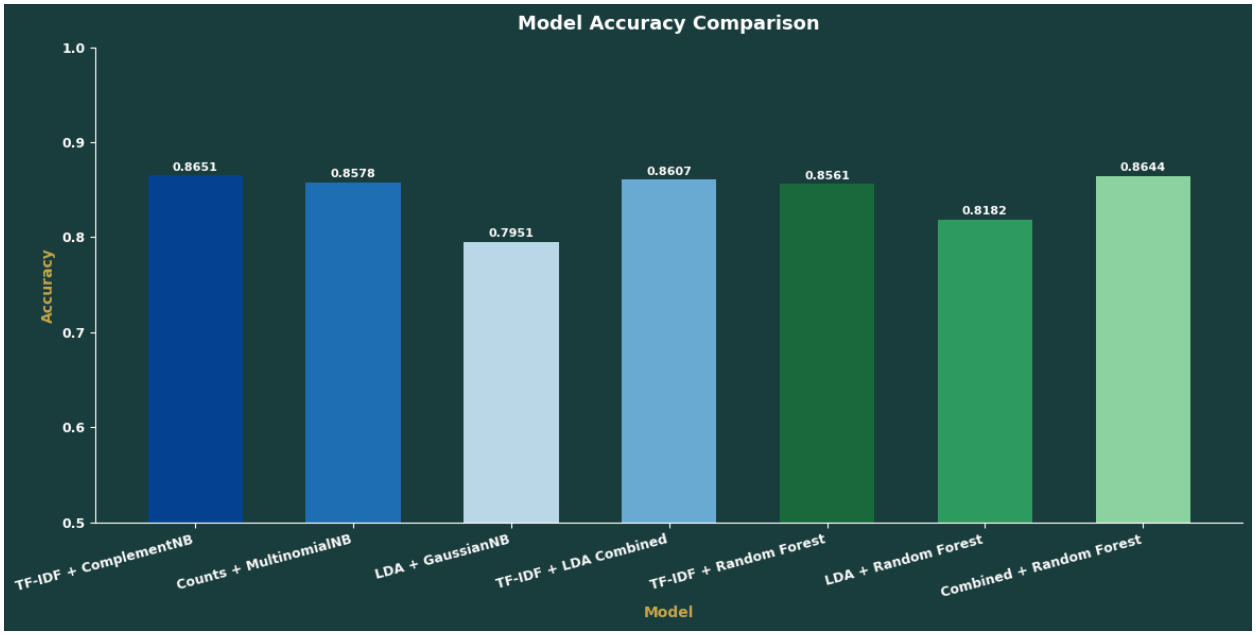

Figure 2: Accuracy comparison across all seven models. Blue shades = Naive Bayes family; green shades = Random Forest family.

5.2 Confusion Matrices — Naive Bayes

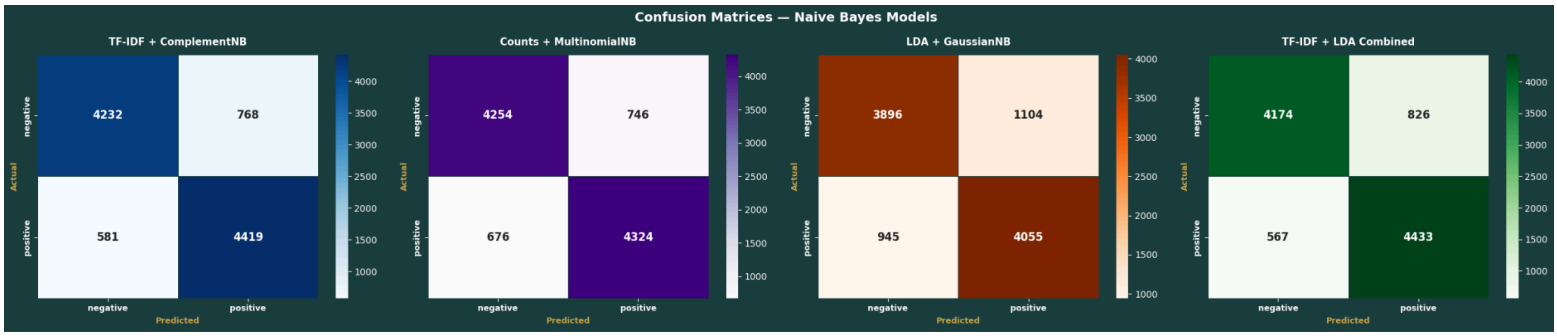

Figure 3: Confusion matrices for all four Naive Bayes models. Rows = actual label, columns = predicted label.

5.3 Confusion Matrices — Random Forest

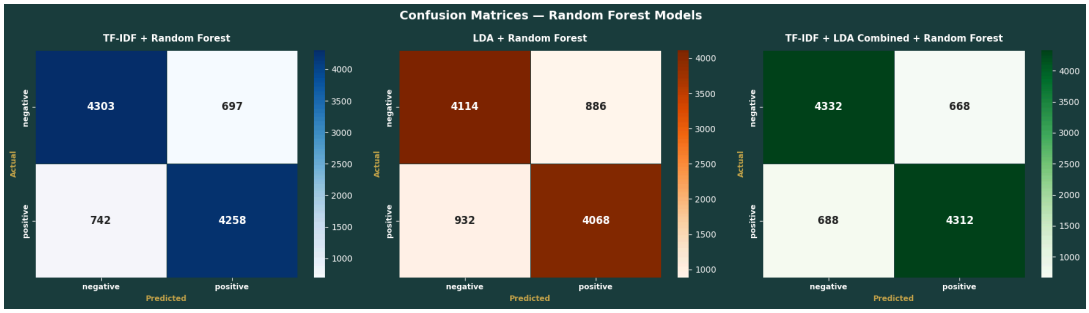

Figure 4: Confusion matrices for all three Random Forest models.

5.4 Vectorizer Comparison

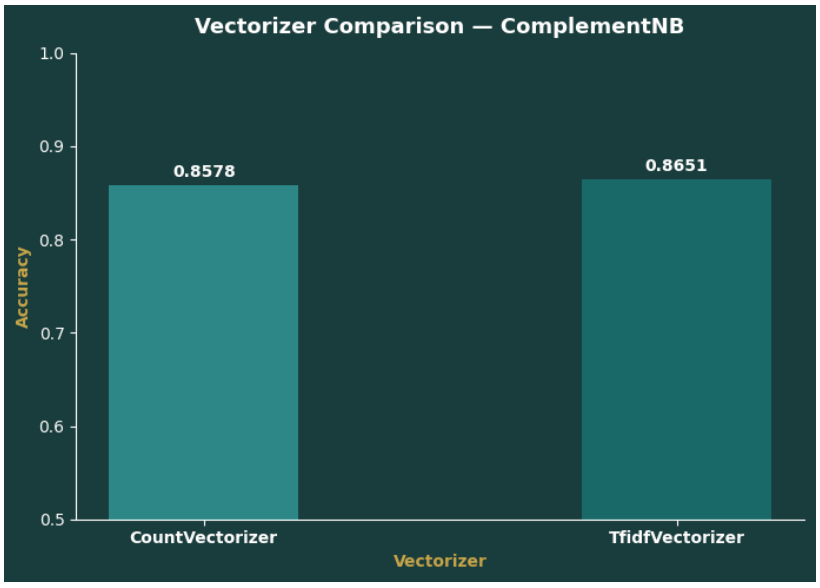

Figure 5: CountVectorizer vs. TfidfVectorizer accuracy using ComplementNB as the fixed classifier.

Both vectorizers perform similarly (CountVectorizer: 85.78%, TfidfVectorizer: 86.51%), with TF-IDF holding a small edge. Notably, we used both in the project not merely to compare them, but because different classifiers require different input types: MultinomialNB requires integer counts (CountVectorizer) while ComplementNB is better suited to TF-IDF float weights. The vectorizer choice was therefore also driven by classifier compatibility, not just performance.

6. Discussion

What We Learned

The most striking finding of this project is that simpler methods outperformed more complex combinations in several key comparisons. TF-IDF with ComplementNB (86.51%) - a simple, well-understood baseline - matched or exceeded most of the more elaborate feature combinations. Adding LDA features to the Naive Bayes pipeline marginally trailed in performance (86.07% combined vs. 86.51% TF-IDF alone), suggesting that GaussianNB-style LDA features introduce noise rather than signal when concatenated with TF-IDF in a Naive Bayes framework.

Random Forest told a different story: the combined TF-IDF + LDA features achieved the best overall accuracy at 86.44%, marginally beating TF-IDF alone (85.61%) with Random Forest. This suggests Random Forest is better equipped to identify which of the 10,010 combined features are informative, while Naive Bayes is diluted by the addition of low-signal LDA columns.

LDA was most valuable not as a classification feature but as an interpretive tool. The 10 topics discovered - covering horror, comedy, musicals, war films, family dramas, and more - demonstrate that reviewers organize their language around genre and narrative theme in consistent, detectable patterns. Whether these themes correlate with sentiment (e.g., horror reviews skewing negative, comedy reviews skewing positive) is an open question worth future investigation.

What More We Could Do

- Aspect-based sentiment analysis: rather than predicting a single positive/negative label per review, use the LDA topics to identify which aspect of a film (acting, plot, visuals, music) the sentiment applies to. This would produce richer, more actionable outputs - telling a filmmaker not just "reviews are mixed" but "reviews are positive about the score but negative about the pacing."
- Hyperparameter tuning for Random Forest: we used n_estimators=100 as a default. A grid search over max_depth, min_samples_split, and n_estimators could meaningfully improve performance. Similarly, the number of LDA topics (10) was chosen with reference to a perplexity curve but not exhaustively optimized.
- Boosting and bagging: ensemble methods like XGBoost or AdaBoost on TF-IDF features would be natural next steps. Bagging with non-tree base learners on LDA features could also improve on GaussianNB's 79.51%.
- Transformer-based models: BERT or RoBERTa fine-tuned on this dataset would likely push accuracy well above 90%, as they capture contextual meaning that bag-of-words representations fundamentally cannot. The gap between our best model (86.44%) and a fine-tuned transformer represents the ceiling we are leaving on the table by using classical ML.
- Applying this to Letterboxd: the IMDB dataset is a useful benchmark, but Letterboxd reviews have a distinct tone - more informal, ironic, and cinephile-specific. Training on Letterboxd data

and evaluating whether the same pipeline generalizes (or requires retraining) would be a compelling real-world extension.

How We Would Inform Others

For someone new to this problem, we would emphasize three things. First, text cleaning is more important than model choice - the regex pipeline that strips HTML and lemmatizes tokens has a larger impact on quality than the difference between ComplementNB and Random Forest. Second, feature representation matters more than classifier sophistication: TF-IDF + a simple Naive Bayes classifier outperforms LDA + Random Forest, which is a useful counterintuitive lesson about not assuming that more complex is better. Third, LDA is worth including even when it does not improve classification accuracy, because the topics it produces are interpretable and tell you something about your data that classification metrics alone cannot.

7. Other Aspects of Machine Learning Used

Beyond the core classification pipeline, this project drew on several additional areas of machine learning:

- Unsupervised learning: LDA is an unsupervised generative model. No labels were used during topic fitting - the algorithm discovered structure in the text entirely from word co-occurrence patterns.
- Dimensionality reduction: LDA also functions as a dimensionality reduction step, compressing a 10,000-dimensional sparse vocabulary into a 10-dimensional dense representation. We explored TruncatedSVD as an alternative reduction approach for enabling LDA/QDA classifiers on TF-IDF.
- Ensemble methods: Random Forest is a bagging ensemble. Understanding when ensembles add value over single models (they do for combined features, but not for TF-IDF alone) was a key analytical result.
- Feature engineering: constructing, combining, and comparing multiple feature representations (raw counts, TF-IDF weights, topic proportions, and stacked combinations) was central to the project design.
- Model evaluation: we used accuracy, precision, recall, F1-score, and confusion matrices to evaluate all models, and a perplexity curve to guide LDA topic selection.

8. Appendix

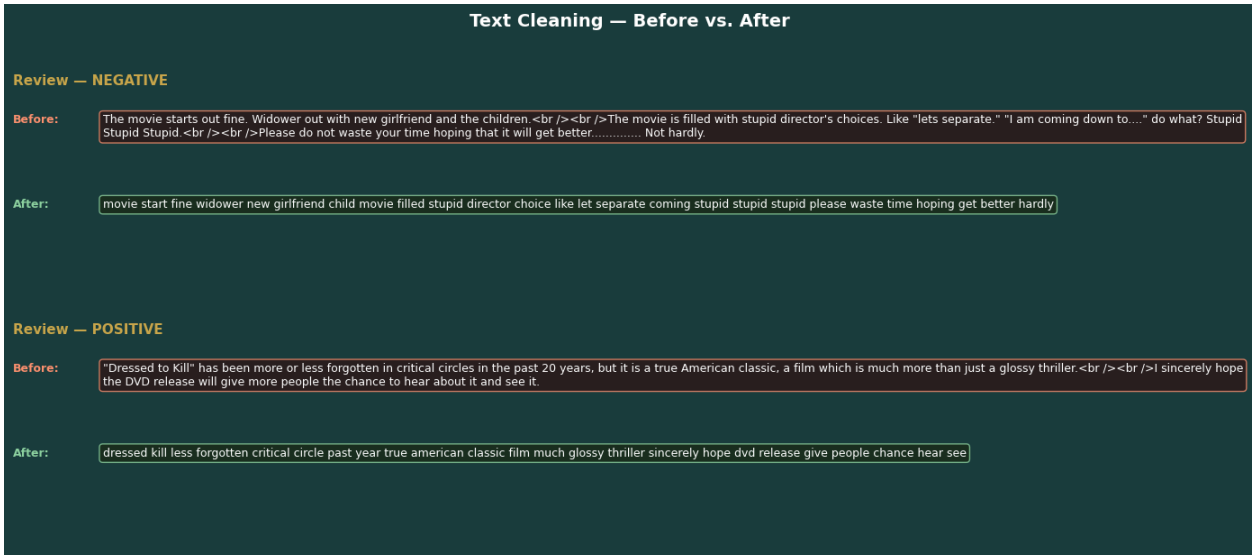

Figure 6: Text Cleaning Pipeline - Before and After. Raw reviews contain HTML artifacts, punctuation, and stopwords. After cleaning, only lemmatized content words remain.

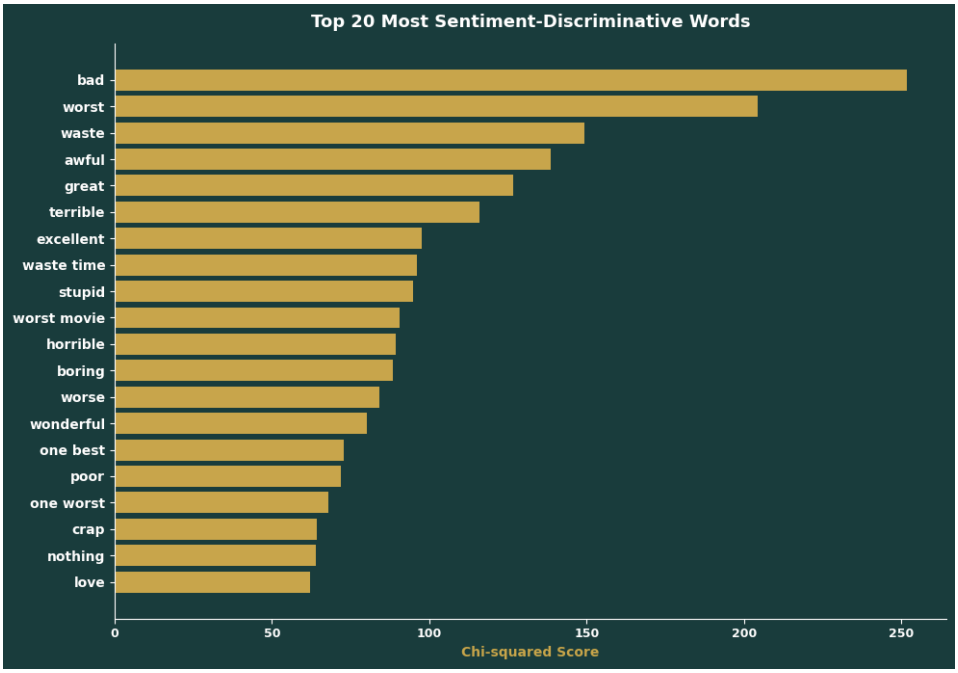

Figure 7: Top 20 Most Sentiment-Discriminative Words by Chi-squared Score. Words are ranked by their association with positive or negative sentiment across all 40,000 training reviews. Bigrams "waste time" and "worst movie" confirm the value of including bigrams in the TF-IDF vocabulary.

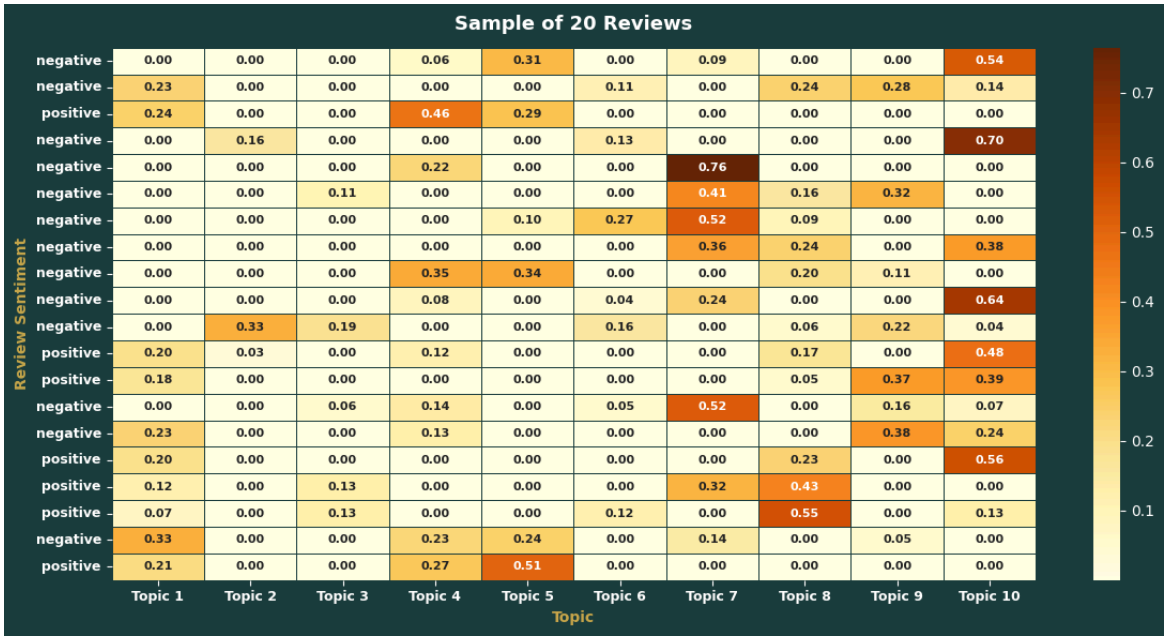

Figure 8: Document-Topic Distribution for a Sample of 20 Reviews. Each row is a review, each column is an LDA topic, and each value represents the proportion of that topic in the review. Rows sum to 1. Darker cells indicate stronger topic membership - reviews are mixtures of topics, not assigned exclusively to one.